Exercise 3 (4 points)

Consider the four snippets of C++ code that follow. Each snippet **may or may not** contain at least one specific problem or include evident bugs. Study each snippet and identify the issue in it, if any. Specifically, observe each line and comment on the bugs you identify, their type, and their effects on the program execution, such as crashes, unexpected output, or memory allocation problems.

Algorithm 1

```
#include <iostream>
#include <vector>

int prefix_sum(const std::vector<int>& v, std::size_t i) {
    if (i == 0) return v[0];
    return prefix_sum(v, i - 1) + v[i];
}

int main() {
    std::vector<int> a = {1, 2, 3};
    std::cout << prefix_sum(a, a.size()) << std::endl;
    return 0;
}
```

Algorithm 2

```
#include <iostream>

int main() {
    int* p = new int[5];
    for (int i = 0; i < 5; ++i)
        p[i] = i;
    delete p;
    return 0;
}
```

Algorithm 3

```cpp
#include <iostream>

void printValue(int* x) {
    std::cout << *x << std::endl;
}

int main() {
    int* ptr = nullptr;
    printValue(ptr);
    return 0;
}
```

Algorithm 4

```cpp
#include <iostream>

struct Base {
    Base()  { std::cout << "Base()\n"; }
    ~Base() { std::cout << "~Base()\n"; }
    virtual void run() { std::cout << "Base::run\n"; }
};

struct Derived : Base {
    Derived()  { std::cout << "Derived()\n"; }
    ~Derived() { std::cout << "~Derived()\n"; }
    void run() override { std::cout << "Derived::run\n"; }
};

int main() {
    Base* p = new Derived();
    delete p;
    return 0;
}
```

rule: the destructor is always virtual

Solution 3

1. **Algorithm 1:** This code contains an **off-by-one bug** in the call `prefix_sum(a, a.size())`. If the vector has size n, valid indices are $0, 1, \ldots, n-1$. Passing `i = n` makes the function access `v[i]` (i.e., `v[n]`), which is **out of bounds** and therefore causes **undefined behavior** (possible crash, garbage output, or seemingly correct output depending on the run).

   Additionally, the function is **inefficient**: computing a prefix sum this way takes $O(i)$ recursive calls and can overflow the stack for large vectors. A simple iterative loop would avoid recursion and be safer.

2. **Algorithm 2:** The array is allocated with `new[]`, but deallocated using `delete` instead of `delete[]`. This is **undefined behavior** and may cause memory corruption, leaks, or crashes.

3. **Algorithm 3:** A null pointer is passed to `printValue` and dereferenced. Dereferencing `nullptr` is **undefined behavior**, typically a runtime crash (segmentation fault).